Multithreading & Multi-Processing

Program → set of instructions written in a programming language

Process → An instance of a program that is being executed
→ One process doesn't corrupt another because seperate memory spaces

Thread → A unit of execution within a process

```
>>> import threading
    import time

    def print_nums():
        for i in range(5):
            time.sleep(2)
            print(f'Number {i}')

    def print_letter()
        for letter in 'abcde':
            print('Letter: {letter}')

    # Create threads
    t1 = threading.Thread(target = print_nums)
    t2 = threading.Thread(target = print_letter)

    t = time.time()          # starting time
    t1.start()
    t2.start()

    t1.join()
    t2.join()

    finished_time = time.time() - t
```

(t1) (t2)

2 functions

Creating threads

Starting both threads

Wait for the threads to complete

→ final time

Multiprocessing

What?

→ Multiple processes running in parallel

Where?

CPU bound tasks

→ eg → maths comp.

Why?

To utilise all CPU cores

```
>>> import multiprocessing
    import time

def f1():
    for i in range(5):
        time.sleep(2)
        print(i)

def f2():
    for i in range(5):
        time.sleep(2)
        print(i*i)

>>> if __name__ == '__main__':
    p1 = multiprocessing.Process(target=f1)
    p2 = multiprocessing.Process(target=f2)

    p1.start()
    p2.start()

    p1.join()
    p2.join()

    finished_time = time.time() - t
```

f1

f2

creating 2 processes

starting the 2 processes

waiting for processes to finish

Threads → share resources like memory etc
→ 1 fails, rest fail
→ Share the same variables
→ Do not restart our script

Multi Process → Start a new interpreter
→ Re imports the script

Imp → Multiprocessing can only be done in a fresh python module, not jupyter notebook.

→ Multithreading can be done either way

Thread pool / Executor

```
>>> from concurrent.futures import ThreadPoolExecutor
def f1(a):
    ~~def f2(a):~~ — not needed
nums = [1, 2, 3, 4, 5, 6, . . . . ]
with ThreadPoolExecutor(max_workers=3) as executor:
    results = executor.map(f1, nums)
for result in results:
    print(result)
```

→ parameters list

→ map func

→ function name

for best results, wrap in list()
=> res = list(exec...)

→ max_workers = 3
→ tells max 3 threads run at same time.

☆ ProcessPoolExecutor

```
>>> from concurrent.futures import ProcessPoolExecutor

def f1(num):
    time.sleep(2)
    return num*2

nums = [1,2,3,4,5]

if __name__ == "__main__":
    with ProcessPoolExecutor(max_workers=3) as executor:
        results = list(executor.map(f1, nums))

    for res in results:
        print(res)
```

funct

input list param

Stuff to note

→ during multiprocessing, use `__name__ == "__main__":`

because new process imports the same script

→ without main, infinite recursion.

Notes

ThreadPoolExecutor & ProcessPoolExecutor → T/P.P Exec

→ ~~import~~ from concurrent.futures import T/P.P Exec

→ Create any function. — either returns / or print values.

~~with ThreadPoolExecutor(max_workers=5) as executor:~~

with (Thread / Process) Pool Executor (max_workers = 5) as executor:

result = list (executor.map(f1, list-par))

→ syntax ✓

executor → returns a conceptual iterator

max_workers → max no of process / threads

list → convert to list

→ good practise

map() & filter() always return iterators.

→ if f1 has no returns, results object creation not needed.

So, use a for loop over the list & executor.submit(f1, i)

eg. with T_exc:

for i in nums:
exe.submit(f1, i) ✓✓

Web Scraping

(I/O bound task)

```
>>> from concurrent.futures import ThreadPoolExecutor
>>> ~~import threading~~
    import requests

    urls = ['url1', 2, 3]

>>> from bs4 import BeautifulSoup

    def f1(url):
        response = requests.get(url)
        soup = BeautifulSoup(response.content, 'html.parser')
        print(len(soup.text))

    with ThreadPoolExecutor(max_workers=3) as executor:
        executor.map(f1, urls)
```

safe/good practice → import all the modules.

→ stores the response of the url request.

→ converts it into soup using html.parser, on response.content.

→ soup.text → has all the text